Real Estate Management System (REMS)

Presented By

Student 1: Midhaat Azad, 214315162
Student 2: Parsa Hemmati, 220918306
Student 3: Komal Dhawan, 220486114

Team Number: 5
Date: November 16, 2025

Table of Contents

1. Introduction.....3
1.1 Motivation.....3
1.2 Project Description.....3
1.3 Organization of the Report.....4
2. Problem Statement and Design Objectives.....5
2.1 Problem Statement and Proposed Solution.....5
2.2 Design Objectives.....5
2.3 Limitations.....5
3. System Specification.....6
3.1 Design Objectives.....6
3.2 Functional Requirements.....6
3.3 Non-Functional Requirements.....6
3.4 Constraints.....7
4. Use Cases.....7
4.1 Use Case Diagram.....8
4.2 Use Case Descriptions.....8
5. User Stories.....12
5.1 Unit Search and Viewing.....12
5.2 Lease Application and Management.....12
5.3 Billing and Reporting.....13
6. Project Management Plan.....13
6.1 Team Roles (Scrum).....13
6.2 Communication Plan.....14
6.3 Backlog Summary.....17
6.4 Group Meeting Logs.....18
7. Testing and Performance Analysis Plan.....19
References.....22

Introduction

Mall property teams manage a diverse portfolio of retail properties and currently use spreadsheets and emails to manage retail tenants, utility systems, maintenance operations, and lease agreements. As their portfolio changes and grows, so do the operational costs to manage these properties. To stay competitive in an industry that is evolving with tech integration and remain an appealing place for tenants to lease mall property management teams need to ensure they provide tenants with easy to access resources while also streamlining their own processes to minimize error and set the roots for growth and profitability.

1.1 Motivation

Real estate development companies managing large-scale shopping malls currently operate with fragmented, manual processes. Property managers are currently juggling spreadsheets to track store availability across multiple locations, which can lead to data inconsistencies and human error. Leasing agents coordinate viewing appointments through email, resulting in double-bookings and frustrated future tenants. Rent collection relies on manual invoicing and payment tracking, which can cause delayed cash flow and strain tenant relationships. Maintenance requests arrive via phone calls and emails without systematic prioritization, leading to emergency issues being overlooked while routine tasks consume staff time and resources. Utility billing requires manual meter readings and separate invoicing, creating administrative burden and billing disputes.

While generic property management systems exist they are often expensive and require a lot of manual customization to fit the unique needs of the real estate development company for shopping mall specific real estate portfolios.

1.2 Project Description

After analyzing the problem landscape we have deducted the following requirements for the solution to be effective and efficient:

1. Property & Lease Management: Track store units by location, size, rental rate, classification tier, and intended business purpose. This will help leasing agents and administrative staff effectively manage the units.
2. Tenant Self-Service Portal: Enable prospective tenants to search available spaces using configurable filters, and schedule conflict-free viewing appointments.
3. Automated Financial Operations: Support flexible payment cycles (monthly, quarterly, bi-annual, annual) with variable pricing, multi-store discount logic, automated invoice generation, payment tracking, and overdue notifications.
4. Integrated Utility Billing: Track and invoice electricity, water, and waste management charges, consolidating them into unified monthly statements alongside rent. Tenants can also access their consumption data which will be available through simulated data sources.

5. Maintenance Management: Provide a request portal with automated prioritization, emergency escalation, first-come-first-served scheduling for routine maintenance which will be offered at no cost, and conditional billing for tenant-caused damage.

As a result of this system functionality the main benefits of this system will be:

1. Operational Efficiency: This software will eliminate manual data entry, which in turn will reduce administrative overhead, and prevent scheduling conflicts through automation.
2. Improved Cash Flow: Accelerate rent collection with automated invoicing, multiple payment options, and proactive overdue reminders.
3. Enhanced Tenant Experience: Provide 24/7 self-service access for searching properties, booking viewings, paying rent, and requesting maintenance. This will improve satisfaction and retention across rental units.
4. Data-Driven Insights: Centralize property, financial, and operational data to enable analytics on occupancy rates, revenue trends, and maintenance patterns.
5. Scalability: It will support portfolio growth across multiple mall properties with consistent workflows and centralized oversight.

REMS will be built on a modern web architecture using **Ruby on Rails** for the backend API and business logic, **React** for the responsive frontend interface, and **PostgreSQL** as the relational database. This stack ensures rapid development, maintainability, and the ability to scale as the company's property portfolio expands.

There will be three main roles/stakeholders:

- **Tenants** – search units, book viewings, apply for leases, pay rent, and submit maintenance requests.
- **Clerks** – approve applications, create leases, generate invoices, track payments.
- **Admins** – view occupancy and revenue reports and oversee all mall operations.

1.3 Organization of the Report

The report is organized as follows:

- Section 2 defines the problem and objectives
- Section 3 details requirements
- Section 4 presents use cases
- Section 5 lists user stories
- Section 6 outlines project management
- Section 7 explains testing and performance plans

Problem Statement and Design Objectives

2.1 Problem Statement and Proposed Solution

Problem

Analyzing the problem we can deduce three large business problems that effect all the stakeholders mentioned above that need to be addressed:

1. Lost Revenue: Vacant units remain unleased and as a result rental income is delayed. With errors or missed billings for utility and maintenance charges.
2. Operational Inefficiency: Staff spend excessive time on manual tasks rather than strategic leasing activities, along with no prioritization system for maintenance tasks.
3. Poor Tenant Experience: Slow responses, booking conflicts, and billing errors drive tenants to competitors

Proposed Solution

The **Real Estate Management System (REMS)** will be a web platform to streamline the processes of tenants, and operators. Features include automated invoices, secure authentication, and clear separation of tenant, clerk, and admin tasks (refer to the project description for system requirements). With the objective to reduce manual workload, provide 24/7 resources to both tenants and operators and a clear paper trail for all the users.

2.2 Design Objectives

1. Provide an online search for available units.
2. Appointment Scheduling System: To prevent booking conflicts for viewings and giving leasing agents a calendar view of appointments.
3. Automate lease creation and invoice generation.
   Track rent payments and maintenance requests.
   Support secure login and role-based access.

2.3 Limitations

The limitations of this software are outlined as the following:

1. This prototype will cover one mall at this moment in time.
2. The payment gateway will be simulated due to account requirements. Payment workflows will be functional but use mock confirmations.
3. Mobile View and Analytics is planned for later phases due to time and resource constraints.
4. The system will not automatically verify legal compliance of the leases. All the legal processes will still be a manual process.

5. There will be no version control for documents, a user will not be able to see historical documents.

System Specification

3.1 Design Objectives

The Real Estate Management System (REMS) is designed to achieve the following objectives:

1. Efficient Property & Lease Management – Enable management of units, leases, and rental payments.
2. Tenant Self-Service – Allow tenants to search units, book viewings, apply for leases, pay rent, and submit maintenance requests.
3. Automation of Financial Operations – Automatically generate invoices, track payments, and send overdue reminders.
4. Maintenance Management – Track and prioritize maintenance requests, support conditional billing.
5. Reporting & Insights – Provide dashboards for occupancy, revenue, and open tickets for clerks and admins.
6. Secure Access Control – Role-based access for tenants, clerks, and admins.

3.2 Functional Requirements

ID	Priority	Requirement	Linked User Story
FR-01	High	Display list of all units with ID, size, rate, status	US-01
FR-02	High	Filter and search units by price or availability	US-01
FR-03	High	Book viewing appointments; prevent overlaps	US-02
FR-04	High	Submit lease applications	US-03
FR-05	High	Approve or reject applications	US-04
FR-06	High	Create and manage leases	US-04
FR-07	Medium	Auto-generate invoices monthly	US-05
FR-08	High	Record payments and update invoice status	US-05
FR-09	Medium	Create and update maintenance tickets	US-02, US-03
FR-10	Medium	Show admin reports (occupancy, income, open tickets)	US-06

3.3 Non-Functional Requirements

ID	Priority	Requirement	Description
NFR-01	High	Performance	API responses should be < 2 seconds under expected load.

NFR-02	Critical	Security	Role-based access, BCrypt password hashing, JWT authentication.
NFR-03	High	Reliability	Ensure transactional consistency in PostgreSQL; uptime ≥ 99%.
NFR-04	Medium	Scalability	Support horizontal scaling for Rails API + React frontend.
NFR-05	High	Maintainability	MVC architecture with automated tests (Minitest + Jest).
NFR-06	High	Interface	Responsive UI with React; intuitive navigation for all roles.
NFR-07	High	Safety	Prevent invalid data entries and enforce validation rules.

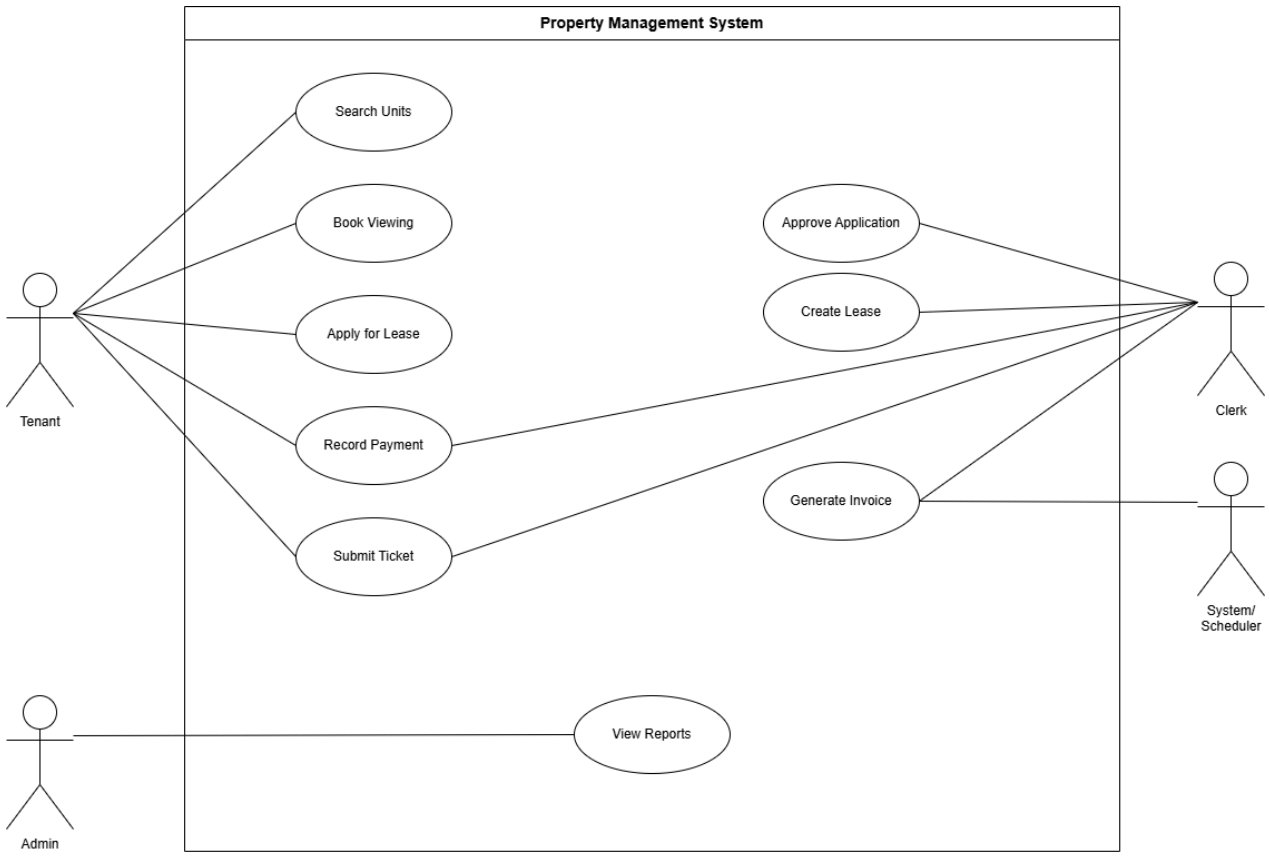
3.4 Constraints

- Languages: Ruby 3.3, JavaScript ES6+ (React 18)
- Frameworks: Rails 7.2 (API mode), React (Vite or Next frontend)
- Database: PostgreSQL 15
- Version Control: GitHub
- Testing: Minitest (Rails), Jest + RTL (React)
- Other: Mock payment gateway; mobile views and analytics deferred to later phases

Use Cases

4.1 Use Case Diagram

Actors: Tenant, Clerk, Admin, System/Scheduler
Use Cases: Search Units, Book Viewing, Apply for Lease, Approve Application, Create Lease, Generate Invoice, Record Payment, Submit Ticket, View Reports



4.2 Use Case Descriptions

UC-01: Book Viewing
Actors: Tenant
Description: This use case describes how tenants schedule property viewing appointments for available units in the system.
Preconditions: Tenants must be logged in and units must be available for viewing.
Main Flow:

1. The tenant navigates to the available units page
2. The tenant selects a desired unit and viewing time slot
3. The system validates availability via Rails API
4. The system saves the appointment record to PostgreSQL database
5. The system displays confirmation to the tenant

Alternate Flow:

- If the selected time slot is already taken, the API returns an error
- The React interface shows the next available time slots to the tenant
- The tenant can select an alternative time

Postconditions: The appointment is stored in the database and is visible to the Clerk in the management dashboard.

UC-02: Apply for Lease

Actors: Tenant, Clerk
Description: This use case describes how tenants submit lease applications for units they are interested in renting.

Preconditions: Tenant accounts must be active.
Main Flow:
1. The tenant selects "Apply for Lease" for a specific unit
2. The tenant fills out the application form with required information
3. The tenant submits the completed application
4. The React frontend posts the data to the Rails API
5. The system saves the application record to the database
6. The Clerk reviews the application
7. The Clerk approves the application
8. The system creates a lease linked to the tenant and unit

Alternate Flow:
- If required fields are missing, the system displays client-side validation errors
- The tenant must correct the errors before resubmitting

Postconditions: The application is linked to the tenant and unit with status set to "Pending" until clerk approval. After approval, a lease is created.

UC-03: Generate Monthly Invoices

Actors: Clerk (System/Scheduler)
Description: This use case describes how the system automatically generates monthly rent invoices for all active leases.
Preconditions: Active lease must exist in the system.
Main Flow:
1. The Rails cron job scheduler runs on the first day of each month
2. The system identifies all active leases in the database
3. The system creates invoice records for each active lease
4. The invoices are saved to the PostgreSQL database
5. The system makes invoices visible in tenant dashboards

Alternate Flow:
- If a lease has ended, the system skips invoice generation for that lease
- Clerk can manually trigger invoice generation if needed

Postconditions: Invoices are ready for tenant viewing in their dashboard and Clerk can track payment status.

UC-04: View Reports

Actors: Admin
Description: This use case describes how administrators access and view system-wide reports including occupancy rates, revenue, and operational metrics.

Preconditions: Admin must be logged in to the system.
Main Flow:
1. The admin selects "View Reports" from the main menu
2. The admin chooses the type of report (occupancy, revenue, maintenance, etc.)
3. The Rails API aggregates data from the PostgreSQL database
4. The system processes and formats the data
5. The React frontend renders interactive charts and visualizations
6. The admin can filter by date range or property

Alternate Flow:
- If no data exists for the selected period, the system displays a "No data available" message
- Admin can export reports to PDF or CSV format

Postconditions: Reports are displayed on the admin dashboard with current data reflecting all system activity.

UC-05: Search Units

Actors: Tenant
Description: This use case describes how tenants search and filter available units to find rental spaces that meet their requirements.
Preconditions: Tenants must be logged in to the system.
Main Flow:
1. The tenant navigates to the "Available Units" page
2. The system displays a list of all available units with ID, size, rate, and status
3. The tenant applies filters (price range, size, availability status)
4. The React frontend sends filter parameters to the Rails API
5. The system queries the PostgreSQL database with filter criteria
6. The system returns filtered results to the frontend
7. The tenant views the filtered list of units

Alternate Flow:
- If no units match the filter criteria, the system displays "No units found" message
- The tenant can clear filters to view all available units again

Postconditions: The tenant sees a filtered list of units matching their search criteria and can proceed to book viewings or apply for leases.

UC-06: Record Payment

Actors: Tenant, Clerk
Description: This use case describes how rent payments are recorded in the system and invoice statuses are updated.
Preconditions: Invoice must exist in the system for an active lease.

Main Flow:
1. The tenant navigates to their "Invoices" dashboard
2. The tenant selects an unpaid invoice
3. The tenant enters payment information (simulated payment gateway)
4. The system validates payment details
5. The Rails API records the payment in the PostgreSQL database
6. The system updates the invoice status to "Paid"
7. The system displays payment confirmation to the tenant
8. The Clerk can view updated payment status in their dashboard

Alternate Flow:
- If payment validation fails, the system displays an error message
- The tenant must re-enter payment information
- Clerk can manually record payments received outside the system (check, wire transfer)

Postconditions: The payment is recorded in the database, invoice status is updated to "Paid," and both tenant and clerk can view the updated status.

UC-07: Submit Maintenance Ticket

Actors: Tenant, Clerk
Description: This use case describes how tenants submit maintenance requests for issues in their leased units.
Preconditions: Tenants must be logged in and have an active lease.
Main Flow:
1. The tenant navigates to the "Maintenance Requests" page
2. The tenant selects "Submit New Ticket"
3. The tenant fills out the maintenance request form (issue description, priority, location)
4. The tenant submits the completed form
5. The React frontend posts the data to the Rails API
6. The system saves the ticket record to the PostgreSQL database
7. The system assigns a ticket ID and status "Open"
8. The system displays confirmation with ticket number to the tenant
9. The Clerk can view the ticket in their maintenance dashboard

Alternate Flow:
- If required fields are missing, the system displays validation errors

- The tenant must complete all required fields before submitting
- For emergency issues, the system automatically escalates priority

Postconditions: The maintenance ticket is stored in the database with status "Open" and is visible to the Clerk for assignment and tracking.

User Stories

5.1 Unit Search and Viewing

1. User Story 1: Search Available Units (US-01)

- **As a tenant**, I want to search units so that I can find available spaces.
- **Acceptance Criteria:**
  - The system displays search filters for unit attributes (size, price, location)
  - Results match the applied filters accurately
  - Only available units are shown in search results
  - Search results update dynamically as filters change
- **Priority:** High
- **Story Points:** 3

2. User Story 2: Book Unit Viewing (US-02)

- **As a tenant**, I want to book a viewing so that I can see a unit.
- **Acceptance Criteria:**
  - The system displays available viewing time slots for selected units
  - Users can select a date and time for viewing
  - Rails validates and rejects overlapping viewing appointments
  - Confirmation is sent upon successful booking
- **Priority:** High
- **Story Points:** 4

5.2 Lease Application and Management

3. User Story 3: Apply for Lease (US-03)

- **As a tenant**, I want to apply for a lease so that I can rent a unit.
- **Acceptance Criteria:**
  - The system provides an application form with required fields (personal info, employment, references)
  - Application is stored in the database upon submission
  - Application status is set to "Pending" automatically
  - User receives confirmation of application submission
  - Input validation ensures all required fields are completed
- **Priority:** High
- **Story Points:** 5

4. User Story 4: Approve Lease Applications (US-04)

- **As a clerk**, I want to approve applications so that I can create leases.
- **Acceptance Criteria:**
  - The system displays a list of pending applications with filtering options
  - Clerk can review complete application details and supporting documents
  - Upon approval, a lease record is created automatically in the database
  - Application status updates to "Approved" and is timestamped
  - Applicant receives automated notification of approval decision
- **Priority:** High
- **Story Points:** 5

5.3 Billing and Reporting

5. User Story 5: Generate Monthly Invoices (US-05)

- **As a clerk**, I want to generate invoices so that rent is billed monthly.
- **Acceptance Criteria:**
  - The system automatically generates invoices on the first of each month for all active leases
  - Invoices include lease details, tenant information, rent amount, and payment due date
  - Invoices are stored in the database and accessible to tenants
  - Tenants can view and download their invoices in PDF format
  - System logs all invoice generation activities
- **Priority:** High
- **Story Points:** 5

6. User Story 6: View Performance Reports (US-06)

- **As an admin**, I want to view reports so that I can track performance.
- **Acceptance Criteria:**
  - The system provides a dashboard with key performance metrics (occupancy rate, revenue, pending applications)
  - Dashboard loads and displays data under 2 seconds
  - Reports can be filtered by custom date ranges
  - Data visualizations (charts, graphs) are clear, accurate, and interactive
  - Dashboard updates in real-time or with manual refresh
- **Priority:** Medium
- **Story Points:** 3

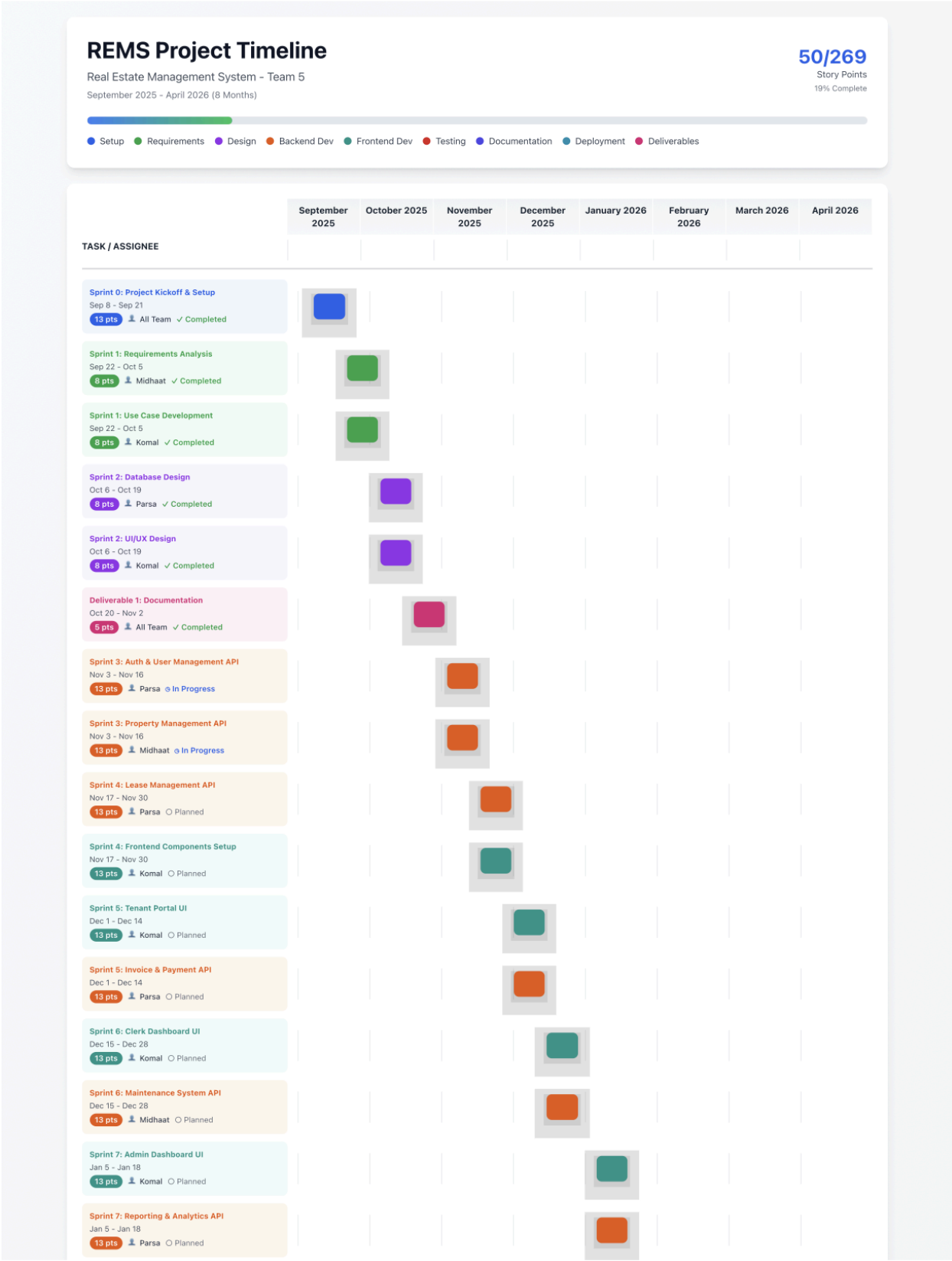
Project Management Plan

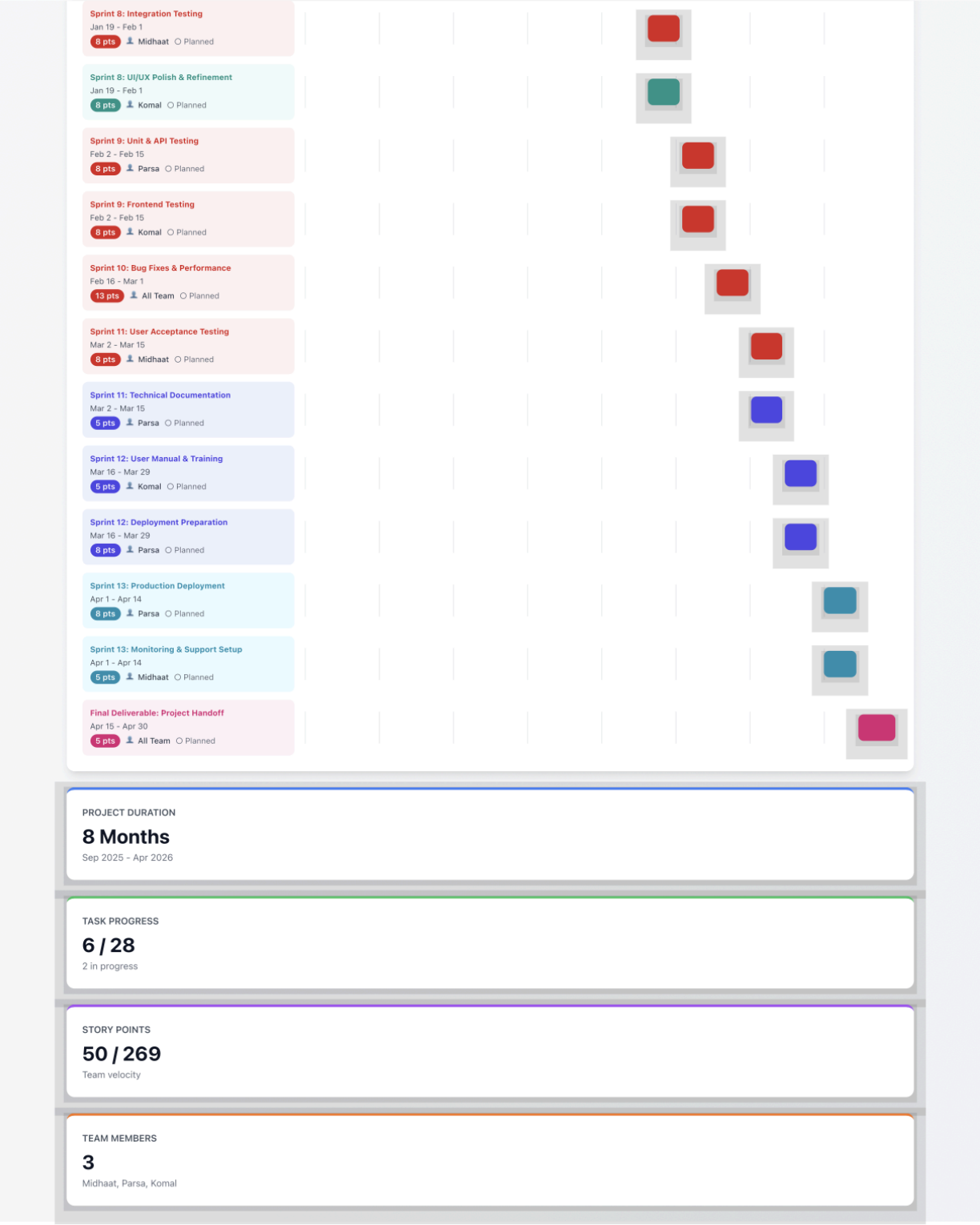
6.1 Team Roles (Scrum)

• **Midhaat Azad** – Product Owner & QA Lead
• **Parsa Hemmati** – Scrum Master & Backend (Rails API)
• **Komal Dhawan** – Frontend Developer (React UI + Design)

6.2 Communication Plan

Our tasks will be defined as issues on a github project board. We will have weekly sprint planning meetings to discuss progress, goals, action items and any blockers. This is paired with async standup at the end of the week to track weekly progress. Meeting minutes will be available on google docs.


REMS Project Timeline
Real Estate Management System - Team 5
September 2025 - April 2026 (8 Months)
50/269
Story Points
19% Complete
Setup
Requirements
Design
Backend Dev
Frontend Dev
Testing
Documentation
Deployment
Deliverables
September 2025
October 2025
November 2025
December 2025
January 2026
February 2026
March 2026
April 2026
TASK / ASSIGNEE
Sprint 0: Project Kickoff & Setup
Sep 8 - Sep 21
13 pts All Team ✓ Completed
Sprint 1: Requirements Analysis
Sep 22 - Oct 5
8 pts Midhaat ✓ Completed
Sprint 1: Use Case Development
Sep 22 - Oct 5
8 pts Komal ✓ Completed
Sprint 2: Database Design
Oct 6 - Oct 19
8 pts Parsa ✓ Completed
Sprint 2: UI/UX Design
Oct 6 - Oct 19
8 pts Komal ✓ Completed
Deliverable 1: Documentation
Oct 20 - Nov 2
5 pts All Team ✓ Completed
Sprint 3: Auth & User Management API
Nov 3 - Nov 16
13 pts Parsa In Progress
Sprint 3: Property Management API
Nov 3 - Nov 16
13 pts Midhaat In Progress
Sprint 4: Lease Management API
Nov 17 - Nov 30
13 pts Parsa Planned
Sprint 4: Frontend Components Setup
Nov 17 - Nov 30
13 pts Komal Planned
Sprint 5: Tenant Portal UI
Dec 1 - Dec 14
13 pts Komal Planned
Sprint 5: Invoice & Payment API
Dec 1 - Dec 14
13 pts Parsa Planned
Sprint 6: Clerk Dashboard UI
Dec 15 - Dec 28
13 pts Komal Planned
Sprint 6: Maintenance System API
Dec 15 - Dec 28
13 pts Midhaat Planned
Sprint 7: Admin Dashboard UI
Jan 5 - Jan 18
13 pts Komal Planned
Sprint 7: Reporting & Analytics API
Jan 5 - Jan 18
13 pts Parsa Planned


Sprint 8: Integration Testing
Jan 19 - Feb 1
8 pts Midhaat Planned
Sprint 8: UI/UX Polish & Refinement
Jan 19 - Feb 1
8 pts Komal Planned
Sprint 9: Unit & API Testing
Feb 2 - Feb 15
8 pts Parsa Planned
Sprint 9: Frontend Testing
Feb 2 - Feb 15
8 pts Komal Planned
Sprint 10: Bug Fixes & Performance
Feb 16 - Mar 1
13 pts All Team Planned
Sprint 11: User Acceptance Testing
Mar 2 - Mar 15
8 pts Midhaat Planned
Sprint 11: Technical Documentation
Mar 2 - Mar 15
5 pts Parsa Planned
Sprint 12: User Manual & Training
Mar 16 - Mar 29
5 pts Komal Planned
Sprint 12: Deployment Preparation
Mar 16 - Mar 29
8 pts Parsa Planned
Sprint 13: Production Deployment
Apr 1 - Apr 14
8 pts Parsa Planned
Sprint 13: Monitoring & Support Setup
Apr 1 - Apr 14
5 pts Midhaat Planned
Final Deliverable: Project Handoff
Apr 15 - Apr 30
5 pts All Team Planned
PROJECT DURATION
8 Months
Sep 2025 - Apr 2026
TASK PROGRESS
6 / 28
2 in progress
STORY POINTS
50 / 269
Team velocity
TEAM MEMBERS
3
Midhaat, Parsa, Komal

6.3 Backlog Summary

AGILE SPRINT BACKLOG - Deliverable 1

BACKLOG TASK & ID	STORY POINTS	ASSIGNED TO	STATUS	ORIGINAL ESTIMATE	DAY 1	DAY 2	DAY 3	DAY 4	DAY 5	SPRINT REVIEW
US-01: Search Units (3 pts)	**3**	Komal/Parsa	To do	4						
Task 1.1: **Backend Core Setup:** Define Unit & User models in PostgreSQL and create authenticated API endpoints for fetching all unit data (ID, size, rate, status, etc.) (FR-01).	1	Parsa	To do	1						
Task 1.2: **Frontend Display:** Build the responsive React component to display the list of units retrieved from the API, ensuring a clear and organized view.	1	Komal	To do	1						
Task 1.3: **Filtering Logic:** Implement the advanced API search functionality to filter units by price range and availability status (FR-02), and integrate the filter controls into the React UI.	1	Komal/Parsa	To do	1						
Task 1.4: **Project Structure QA:** Write basic smoke tests for the unit API endpoint and ensure the Rails-React stack is stable and version-controlled on GitHub (NFR-05).	1	Midhat	To do	1						
US-02: Book Viewing (4 pts)	**5**	Midhat/Komal	To do	4						
Task 2.1: **Appointment Model & API:** Define the Appointment model, create the API endpoint for booking, and implement security controls for tenant access (NFR-02).	2	Parsa	To do	1						
Task 2.2: **Conflict-Free Logic:** Implement the critical backend validation that rejects new bookings if the selected time slot is already taken for that unit/clerk, enforcing (FR-03).	1	Komal	To do	1						
Task 2.3: **Booking Form UI:** Design and implement the React modal/page for tenants to select available units and specific time slots, providing immediate feedback on availability.	1	Midhat	To do	1						
Task 2.4: **Clerk Viewing Dashboard:** Build a simple Clerk interface (list view) to see all upcoming viewing appointments for their scheduled management.	1	Parsa/Midhat	To do	1						
US-03: Leasing Workflow (10 pts)	**10**	Parsa/Midhat	To do	10						
Task 3.1: **Application Submission:** Create the Application model and API endpoint (FR-04). Develop the comprehensive React application form with client-side validation for required fields (NFR-07).	3	Parsa/Midhat	To do	3						
Task 3.2: **Clerk Approval Dashboard:** Build the dedicated Clerk dashboard view to list all pending applications and link to the action interface (Approve/Reject).	2	Komal	To do	2						
Task 3.3: **Lease Creation & Approval API:** Implement the core business logic in the API to handle the Clerk's approval action (FR-05), which automatically creates a new Lease record in the database (FR-06).	3	Komal	To do	3						
Task 3.4: **Validation & Transition:** Ensure data safety and transactional consistency during the application approval and lease creation process (NFR-03). Write Minitest for the approval logic.	2	Parsa	To do	2						
US-04: Financial & Operations (9 pts)	**9**	Parsa/Komal	To do	9						
Task 4.1: Invoice Automation: Implement the Rails cron job/scheduler to automatically generate and save monthly invoice records for all active leases (FR-07).	3	Midhat	To do	3						
Task 4.2: Payment Recording: Implement the API logic for tenants/clerks to record payments and update the corresponding invoice status to "Paid" (FR-08).	2	Parsa	To do	2						
Task 4.3: Maintenance Ticket System: Create the Ticket model and API (FR-09). Build the Tenant UI form for submitting requests (priority, location).	2	Komal	To do	2						
Task 4.4: Operations Tracking: Build the basic Clerk dashboard to view, track, and update the status/priority of submitted maintenance tickets.	2	Parsa	To do	2						
User Story #5: Reporting & QA (9 pts)	**9**	Komal/Midhat	To do	9						
Task 5.1: Admin Reporting API: Create the backend API endpoints to aggregate data and calculate key metrics: occupancy rates, revenue, and open ticket counts (FR-10).	3	Komal	To do	3						
Task 5.2: Admin Dashboard UI: Build the protected Admin UI to consume the reporting APIs and display interactive charts and visualizations (NFR-06).	2	Midhat	To do	2						
Task 5.3: Unit & Integration Testing: Conduct full-system QA, focusing on transactional logic. Run all unit and integration tests (Minitest, Jest) and resolve critical bugs (NFR-01, NFR-03).	3	Midhat	To do	3						
Task 5.4: Documentation & Handover: Finalize all project documentation, update the report, and prepare for presentation/ submission.	2	Parsa	To do	2						
TOTAL				73	0	0	0	0	0	0

AGILE PRODUCT BACKLOG TEMPLATE

Team	Team 5
Project	Real Estate Management System (REMS)

TASK ID	TASK NAME	SPRINT #	ASSIGNED TO	START	FINISH	STORY	SPRINT READY	PRIORITY	STATUS	STORY POINTS	ASSIGNED TO SPRINT
		Sprint 0 – Project Kickoff	Team 5	10/8/2025	2025-10-15	Yes	Yes	High	Completed	28	Yes
0.01	Team formation & scope alignment	As above	All	10/8/2025	10/8/2025	Yes	Yes	High	Completed	1	Yes
0.02	Project vision statement	As above	All	10/10/2025	10/13/2025	Yes	Yes	High	Completed	2	Yes
0.03	Initial project constraints & assumptions	As above	All	10/14/2025	10/15/2025	Yes	Yes	High	Completed	1	Yes
		Sprint 1 – User Stories & Use Case Brainstorm	Team 5	10/22/2025	2025-10-29	Yes	Yes	High	Completed		Yes
1.01	Brainstorm and list initial user stories	As above	All	10/22/2025	10/25/2025	Yes	Yes	High	Completed	3	Yes
1.02	Identify actors & stakeholders	As above	Komal	2025-10-26	2025-10-27	Yes	Yes	High	Completed	2	Yes
1.03	Draft preliminary use case list	As above	Parsa	2025-10-27	2025-10-28	Yes	Yes	High	Completed	2	Yes
		Sprint 2 – Workshopping Requirements	Team 5	2025-10-29	2025-11-05	Yes	Yes	High	Completed		Yes
2.01	Develop functional requirements FR-01–FR-10	As above	Midhaat	2025-10-29	2025-10-31	Yes	Yes	High	Completed	5	Yes
2.02	Develop non-functional requirements NFR-01–NFR-07	As above	Komal	2025-10-31	2025-11-03	Yes	Yes	High	Completed	3	Yes

AGILE PRODUCT BACKLOG TEMPLATE

Team	Team 5
Project	Real Estate Management System (REMS)

TASK ID	TASK NAME	SPRINT #	ASSIGNED TO	START	FINISH	STORY	SPRINT READY	PRIORITY	STATUS	STORY POINTS	ASSIGNED TO SPRINT
2.03	Create detailed use case diagrams/descriptions	As above	All	2025-11-03	2025-11-05	Yes	Yes	High	Completed	4	Yes
		Sprint 3 – Strategizing Testing	Team 5	2025-11-12	2025-11-19	Yes	Yes	High	Completed		Yes
3.01	Draft testing strategy outline	As above	Parsa	2025-11-14	2025-11-16	Yes	Yes	High	Completed	3	Yes
3.02	Draft performance analysis approach	As above	Midhaat/Komal	2025-11-17	2025-11-19	Yes	Yes	High	Completed	2	Yes
3.03	Prepare Deliverable 1 compilation	As above	All	2025-11-19	2025-11-23	Yes	Yes	High	Completed	3	Yes

Our complete product backlog and sprint backlog can also be found on our github repo.

6.5 Group Meeting Logs

Date	Attendees	Topics	Decisions
Oct 08	All	1. Officially kick off the Real Estate Management System (REMS) project. 2. Establish team roles and expectations. 3. Clarify project scope and identify initial constraints.	1. Use Rails + React + PostgreSQL 2. Identified team rituals 3. Identified product scope
Oct 22	All	1. Brainstorm initial user stories. 2. Identify actors and stakeholders. 3. Prepare for use case diagram development.	1. Identified stakeholders and potential use cases 2. Assigned work to drafting the diagram and writing up the use cases
Oct 29	All	1. Discussed functional and non-functional requirements.	1. Identified format and number of requirements

			2. Assigned work to writing them up and refining the requirements
Nov 12	All	1. Develop testing strategy section. 2. Plan system performance evaluation approach. 3. Deliverable 1 submission	1. Assigned out work to write up for submission 1 deliverable 2. Agreed on a testing strategy 3. Created a rough system performance evaluation approach and assigned it to be refined.

Testing and Performance Analysis Plan

Testing approach:
• Rails Minitest for model and API logic.
• React Jest and RTL for UI validation.
• Integration tests using Capybara.

TC-01 — Book Viewing Appointments

Field	Details
Test Case ID	TC-01
Related Requirement	FR-03 — Book Viewing Appointments
Preconditions	Tenant is logged in; a unit has available time slots
Steps	1. Tenant selects a viewing time slot 2. Tenant attempts to book the same slot again
Expected Result	• System rejects second booking attempt• Conflict message is displayed
Priority	High

TC-02 — Generate Monthly Invoices

Field	Details
Test Case ID	TC-02
Related Requirement	FR-07 — Generate Monthly Invoices
Preconditions	At least one active lease exists
Steps	1. Run automated invoicing job (manual trigger) 2. System checks all active leases
Expected Result	• One invoice generated per active lease

Priority	High

TC-03 — Record Payments

Field	Details
Test Case ID	TC-03
Related Requirement	FR-08 — Record Payments
Preconditions	Unpaid invoice exists for an active lease
Steps	1. Tenant selects an unpaid invoice 2. Enters payment details 3. Confirms payment
Expected Result	• Payment is recorded • Invoice status updates to “Paid”
Priority	Medium

TC-04 — Create/Update Maintenance Tickets

Field	Details
Test Case ID	TC-04
Related Requirement	FR-09 — Maintenance Tickets
Preconditions	Tenant is logged in with an active lease
Steps	1. Tenant opens Maintenance page 2. Submits a new maintenance ticket
Expected Result	• Ticket created with status “Open”
Priority	Medium

TC-05 — Display & Filter Units

Field	Details
Test Case ID	TC-05
Related Requirements	FR-01, FR-02 — Display & Filter Units
Preconditions	Units with various sizes/prices exist
Steps	1. Tenant navigates to Units page 2. Applies price filter: $1000–$2000
Expected Result	• Only units within selected range are displayed
Priority	Medium

TC-06 — Submit Lease Application

Field	Details
Test Case ID	TC-06
Related Requirement	FR-04 — Submit Lease Application

Preconditions	Tenant logged in; selected unit available
Steps	1. Tenant fills lease application 2. Submits form
Expected Result	• Application saved with status “Pending”
Priority	High

TC-07 — View Reports

Field	Details
Test Case ID	TC-07
Related Requirement	FR-10 — View Reports
Preconditions	Admin logged in; relevant data exists
Steps	1. Admin opens Reports dashboard 2. Opens occupancy report
Expected Result	• Dashboard loads in < 2 seconds • Data is correct and up-to-date
Priority	Medium

TC-08 — Approve/Reject Lease Application

Field	Details
Test Case ID	TC-08
Related Requirement	FR-05 — Approve or Reject Applications
Preconditions	Admin is logged in; at least one lease application with status "Pending" exists
Steps	1. Admin navigates to Applications page 2. Selects a pending application 3. Reviews application details 4. Clicks "Approve" or "Reject" button
Expected Result	• Application status updates to "Approved" or "Rejected" • Tenant receives notification of decision
Priority	Medium

TC-09 — Create Lease

Field	Details
Test Case ID	TC-09

Related Requirement	FR-06 — Create and Manage Leases
Preconditions	Admin is logged in; an approved lease application exists for a specific unit
Steps	1. Admin navigates to Leases page 2. Selects "Create Lease" from approved application 3. Enters lease start date, end date, and rent amount 4. Confirms and saves lease
Expected Result	• Lease is created with status "Active" • Unit status updates to "Occupied" • Lease details are saved correctly
Priority	High

Performance goal: All API calls p95 < 500 ms on demo data.

References

- DIGT 3101 Deliverable 1 Description (Fall 2025) DIGT3101_Deliverable1
- Team GitHub Project Board and Meeting Logs (Fall 2025)
- Our GitHub Repo: https://github.com/220918306/DIGT-3101